Chapter 17 — Telemetry, Observability, and Performance Tools

> *"The difference between a 40% MFU cluster and a 65% MFU cluster is usually not hardware. It is the quality of the observability stack that lets engineers see exactly where compute is being wasted. You cannot optimize what you cannot measure."* — Production AI Infrastructure Principle

---

Chapter Overview

A 10,000-GPU training cluster generates thousands of metrics per second from every GPU, NIC, switch, and storage node. Without the right observability stack, a silent performance regression wastes millions of dollars before anyone notices. A driver update that degrades MFU from 52% to 44% on 10,000 H100s costs $47,000/day if undetected — more than the CI pipeline that would catch it in 20 minutes.

This chapter builds the complete observability architecture from silicon counters to user-facing dashboards to automated regression detection.

Sections in this chapter:

- 17.1 The Seven-Layer Observability Architecture
- 17.2 Layer 1 — DCGM: Silicon-Level GPU Telemetry
- 17.3 Layer 2 — Kernel Profiling: Nsight Systems and Nsight Compute
- 17.4 Layer 3–4 — Framework and System Metrics [r3/r5]
- 17.5 Layer 5 — Inference Service Metrics
- 17.6 Layer 6 — Prometheus, Grafana, and Fleet Alerting [r5]
- 17.7 Straggler Detection and Anomaly Patterns [r5]
- 17.8 Layer 7 — Performance Regression CI Pipeline
- 17.9 AMD Observability Stack
- 17.10 Key Takeaways and Review Questions

17.1 The Seven-Layer Observability Architecture

```
COMPLETE AI SYSTEM OBSERVABILITY — SEVEN LAYERS:

┌──────────────────────────────────────────────────────────────────────────┐
```

```
│  LAYER 7: REGRESSION CI/CD PIPELINE                                  │
│  Statistical baselines, automated perf regression on every          │
│  model/driver/framework update. Gate before fleet deployment.      │
├──────────────────────────────────────────────────────────────────────┤
│  LAYER 6: DASHBOARDS & ALERTING                                      │
│  Grafana heatmaps, PagerDuty oncall, Slack notifications            │
│  Thanos/Mimir for long-term (90-day+) metric retention              │
├──────────────────────────────────────────────────────────────────────┤
│  LAYER 5: SERVICE-LEVEL METRICS                                      │
│  TTFT/TPOT histograms, tokens/sec, KV cache %, queue depth          │
│  vLLM Prometheus endpoint, TRT-LLM telemetry, distributed trace     │
├──────────────────────────────────────────────────────────────────────┤
│  LAYER 4: SYSTEM METRICS                                             │
│  Prometheus + node_exporter: CPU, RAM, NIC, IB port errors          │
│  NCCL timing (custom), InfiniBand counters (IB exporter)            │
├──────────────────────────────────────────────────────────────────────┤
│  LAYER 3: FRAMEWORK METRICS                                          │
│  Training MFU, step time, gradient norm, AllReduce latency          │
│  PyTorch Profiler, WandB/MLflow experiment tracking                 │
├──────────────────────────────────────────────────────────────────────┤
│  LAYER 2: KERNEL PROFILING (on-demand, not always-on)               │
│  Nsight Compute (hardware counters), Nsight Systems (timeline)      │
│  Triggered by anomaly detection; NOT run continuously               │
├──────────────────────────────────────────────────────────────────────┤
│  LAYER 1: SILICON — DCGM HARDWARE COUNTERS                           │
│  GPU utilization, temperature, ECC errors, clock, power, NVLink     │
│  Always-on, 1-second sampling, every GPU in the cluster             │
└──────────────────────────────────────────────────────────────────────┘


FOUR OBSERVABILITY SCOPES:
  Fleet:   Thousands of GPUs simultaneously → DCGM + Prometheus + Grafana
  Job:     One training run → MFU/step time logged to WandB/MLflow
  Kernel:  One GPU, one kernel → Nsight Compute (triggered, not always-on)
  Silicon: PCIe, BMC, ECC hardware events → DCGM + out-of-band management

MENTAL MODEL (for interviews):
  "DCGM is the reporter: reads GPU silicon counters."
  "Prometheus is the historian: stores everything, every second."
  "Grafana is the window: shows what Prometheus recorded."
  "Together: complete visibility into what every GPU in a 10,000-node
   cluster is doing at any moment and in any historical window."
```

17.2 Layer 1 — DCGM: Silicon-Level GPU Telemetry

17.2.1 Core DCGM Metrics Reference

```
ESSENTIAL DCGM FIELD IDs AND ALERT THRESHOLDS (H100 SXM5):

  Field ID   Metric Name                      Healthy Range    Alert Threshold
  ──────────────────────────────────────────────────────────────────────────────
  203        DCGM_FI_DEV_GPU_UTIL             > 75% (compute)  < 40% during job
             SM active %                      < 30% (idle OK)  > 95% sustained

  155        DCGM_FI_DEV_POWER_USAGE          80-95% TDP       > 98% or < 50%
             Instantaneous power (W)          (560-665W H100)

  100        DCGM_FI_DEV_GPU_TEMP             < 80°C           > 82°C (WARNING)
             Junction temperature (°C)                         > 83°C (PAGE)

  150        DCGM_FI_DEV_SM_CLOCK             1,785-1,980 MHz  < 1,700 MHz
             SM clock frequency (MHz)                          (throttle signal)

  1001       DCGM_FI_DEV_ECC_SBE_VOL_TOTAL    ~0               > 1/hour (WARNING)
             Correctable ECC errors                            > 10/hour (DRAIN)

  1002       DCGM_FI_DEV_ECC_DBE_VOL_TOTAL    0                ANY non-zero (PAGE)
             Uncorrectable ECC errors                          → GPU unsafe

  1003       DCGM_FI_PROF_GR_ENGINE_ACTIVE    > 60% (training)  < 30% during step
             Graphics engine active %

  1004       DCGM_FI_PROF_PIPE_TENSOR_ACTIVE  > 50% (training)  < 20% (not using
TCs)
             Tensor Core pipeline active %

  1005       DCGM_FI_PROF_DRAM_ACTIVE         40-85% (normal)   > 90% (BW
saturated)
             HBM bandwidth utilization %                         < 10% with active
job

  1009       DCGM_FI_PROF_NVLINK_TX_BYTES     Proportional      > 20% drop from
peer
             NVLink transmitted bytes (B/s)   to TP load        median (link
degraded)
```

```
  1010       DCGM_FI_DEV_THERMAL_VIOLATION    0                 ANY non-zero
             Thermal throttle active (bool)                     → immediate alert
  ──────────────────────────────────────────────────────────────────────────────


DCGM COLLECTION MODES:
  dcgmi dmon -e 203,155,100,1001,1002 -d 1000
    # Continuous monitoring: fields 203,155,100,1001,1002 every 1000ms
    # Runs on every GPU node as a daemon

  dcgmi job --start <JOBID>      # Begin GPU-hours accounting for a SLURM job
  dcgmi job --stop <JOBID>       # End accounting + get aggregate stats

  dcgmi health -g 0 -j           # JSON health summary for GPU group 0
  dcgmi diag -r 1 -j             # Quick diagnostics: memory bandwidth + compute
```

17.2.2 XID Errors — The GPU Incident Response System

```
XID ERROR CLASSIFICATION (most important for production):

  XID    Meaning                          Severity    Action
  ──────────────────────────────────────────────────────────────────────────
  8      GPU Hang                         HIGH        Driver reset; check job
  13     Graphics Engine Fault            HIGH        Driver reset or node reboot
  31     GPU Memory Page Fault            MEDIUM      Check CUDA code; may be app
bug
  48     DBE (Double-Bit ECC Error)       CRITICAL    PAGE immediately; drain node
  74     NVLink Error                     HIGH        Check NVLink topology; may
need FRU
  79     GPU Memory Page Fault            MEDIUM      Usually application;
investigate
  95     Uncontained Error                CRITICAL    PAGE; immediate FRU + node
replace
  ──────────────────────────────────────────────────────────────────────────


MONITORING XID ERRORS:
  # Via dmesg on each node:
  dmesg -T | grep -E "NVRM.*Xid"

  # Via DCGM DCGM_FI_DEV_XID_ERRORS field:
  dcgmi dmon -e 230  # XID error code, latest event

  # Via Prometheus (after DCGM exporter setup):
  # dcgm_xid_errors_total{gpu="0", Hostname="gpu-node-01"} 1
```

```
  # Alert: any XID 48 or 95 → immediate page

ECC CORRECTABLE ERRORS — EARLY WARNING SYSTEM:
  GPU HBM ECC correctable errors (XID 63, DCGM field 1001):
    0 errors/hour: normal (most GPUs throughout their lifetime)
    1-10/hour:     Warning — schedule GPU replacement within 48-72 hours
    > 10/hour:     Urgent — drain node, schedule immediate replacement
    Any uncorrectable (XID 48): Immediate — do not run workloads, replace GPU

  WHY CORRECTABLE ERRORS MATTER:
    Each SBE (single-bit error) is corrected by hardware ECC.
    But correctable error rate predicts uncorrectable error rate:
    GPUs with high SBE rates fail uncorrectable at 10-50× the rate of clean GPUs.
    A sustained SBE rate of 5/hour means ~1% chance of uncorrectable within 24h.
    Pro-active replacement at SBE threshold prevents uncorrectable mid-training.
```

17.2.3 DCGM Deployment Architecture

```yaml
```

```yaml
# dcgm-exporter-daemonset.yaml — Deploy on every GPU node (Kubernetes)
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: dcgm-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: dcgm-exporter
  template:
    metadata:
      labels:
        app: dcgm-exporter
    spec:
      nodeSelector:
        nvidia.com/gpu: "true"           # Only run on GPU nodes
      containers:
        - name: dcgm-exporter
          image: nvcr.io/nvidia/k8s/dcgm-exporter:3.3.5
          ports:
            - containerPort: 9400        # Prometheus scrapes this port
          env:
            - name: DCGM_EXPORTER_LISTEN
              value: ":9400"
            - name: DCGM_EXPORTER_COLLECTORS
              value: "/etc/dcgm-exporter/custom-collectors.yaml"
          resources:
            limits:
              nvidia.com/gpu: 1          # Needs GPU access to read counters
          securityContext:
            privileged: true             # Required for hardware counter access

# custom-collectors.yaml — Enable high-value profiling metrics
# DCGM_FI_PROF_PIPE_TENSOR_ACTIVE, DCGM_FI_PROF_DRAM_ACTIVE, DCGM_FI_PROF_NVLINK_*
```

17.3 Layer 2 — Kernel Profiling: Nsight Systems and Nsight Compute

17.3.1 The Two-Tool Protocol

```
NSIGHT SYSTEMS vs NSIGHT COMPUTE — CHOOSING THE RIGHT TOOL:
```

```
  Nsight Systems (nsys):
    Scope:      Full application timeline (milliseconds to minutes)
    Overhead:   1.5-3× slowdown (always-on event collection)
    Output:     .nsys-rep file (open in Nsight Systems GUI)
    Use for:    "Where does time go in my training step?"
                Timeline of all GPU kernels, CPU dispatch, NCCL, memory copies
    Answer:     Which kernels dominate? Is AllReduce overlapped? Is GPU idle?

  Nsight Compute (ncu):
    Scope:      Single kernel, hardware counter resolution
    Overhead:   100-1000× slowdown (full replay required)
    Output:     .ncu-rep file (open in Nsight Compute GUI)
    Use for:    "WHY is this specific kernel slow?"
                TC utilization, HBM traffic, warp stalls, occupancy
    Answer:     Is it compute-bound? Memory-bound? What stall type?

  THE PROTOCOL — ALWAYS THIS ORDER:
    Step 1: nsys profile (5 minutes) → identify bottleneck kernel
    Step 2: ncu --kernel-name X (20 minutes) → diagnose why that kernel is slow
    NEVER: open ncu without nsys first (blindly profiles wrong kernel)

NSIGHT SYSTEMS — PRODUCTION COMMAND:
  nsys profile \
    --trace=cuda,nvtx,nccl,osrt \   # CUDA kernels + NVTX markers + NCCL + OS
runtime
    --sample=cpu \                   # CPU stack sampling
    --output=/tmp/profile_%p \      # Separate file per process (rank)
    --duration=30 \                  # 30-second capture window
    python train.py

  READING THE TIMELINE (what to look for):
    GPU row: kernel bars should be dense (no gaps) → gaps = overhead bound
    NCCL row: should overlap with backward pass kernels → separated = no overlap
    CPU row: should show minimal Python overhead between GPU launches

NSIGHT COMPUTE — TARGETED KERNEL ANALYSIS:
  ncu \
    --set full \                     # All metrics (use after nsys identifies
target)
    --section SpeedOfLight_RooflineChart \
    --section MemoryWorkloadAnalysis \
    --section WarpStateStatistics \
    --kernel-name "flash_fwd_kernel" \
```

```
    --launch-skip 10 \               # Skip warmup launches
    --launch-count 1 \
    -o /tmp/kernel_profile \
    python workload.py

  KEY ROOFLINE INTERPRETATION:
    Kernel dot BELOW bandwidth roof: memory-bandwidth-bound
      → Check dram__bytes_read vs theoretical minimum
      → Apply: operator fusion, quantization, tile size increase
    Kernel dot NEAR/AT compute roof: compute-bound
      → Check sm__pipe_tensor_cycles_active (TC utilization)
      → If TC util < 50%: alignment issue (pad matrix dimensions)
    Kernel dot BELOW both roofs: overhead/occupancy-bound
      → Check smsp__warp_issue_stall_not_selected (too few warps)
      → Fix: increase block size, reduce register pressure
```

17.4 Layer 3–4 — Framework and System Metrics [r3/r5]

17.4.1 Training Framework Metrics — MFU and Step Time

```python
```

```python
# training_telemetry.py — Production training metrics with Prometheus

from prometheus_client import Gauge, Histogram, Counter, start_http_server
import torch, time, torch.distributed as dist

# ─── METRIC DEFINITIONS ──────────────────────────────────────────────────
training_mfu      = Gauge('training_mfu',      'Model FLOP utilization', ['job_id'])
step_time         = Histogram('training_step_time_seconds', 'Per-step wall clock',
                              ['job_id'], buckets=[0.5, 1.0, 1.5, 2.0, 3.0, 5.0, 10.
forward_time      = Histogram('training_forward_time_seconds', 'Forward pass', ['job
backward_time     = Histogram('training_backward_time_seconds', 'Backward pass', ['j
allreduce_time    = Histogram('training_allreduce_time_seconds', 'AllReduce', ['job_
gradient_norm     = Gauge('training_gradient_norm', 'Gradient L2 norm', ['job_id'])

# ─── INSTRUMENTED TRAINING STEP ──────────────────────────────────────────
def instrumented_step(model, batch, optimizer, job_id, model_params, num_gpus, gpu_p
    t_step_start = time.perf_counter()

    # Forward pass
    t0 = time.perf_counter()
    loss = model(**batch).loss
    torch.cuda.synchronize()
    fwd_ms = (time.perf_counter() - t0) * 1000
    forward_time.labels(job_id=job_id).observe(fwd_ms / 1000)

    # Backward pass
    t0 = time.perf_counter()
    loss.backward()
    torch.cuda.synchronize()
    bwd_ms = (time.perf_counter() - t0) * 1000
    backward_time.labels(job_id=job_id).observe(bwd_ms / 1000)

    # AllReduce timing (DDP/FSDP; barrier before optimizer to isolate)
    t0 = time.perf_counter()
    optimizer.step()
    torch.cuda.synchronize()
    ar_ms = (time.perf_counter() - t0) * 1000
    allreduce_time.labels(job_id=job_id).observe(ar_ms / 1000)
    optimizer.zero_grad()

    total_s = time.perf_counter() - t_step_start
    step_time.labels(job_id=job_id).observe(total_s)
```

```python
    # MFU calculation (Chinchilla 6N formula)
    tokens_per_second = batch['input_ids'].numel() / total_s  # Approximate
    flops_per_token = 6 * model_params
    achieved_flops = tokens_per_second * flops_per_token
    mfu = achieved_flops / (num_gpus * gpu_peak_flops)
    training_mfu.labels(job_id=job_id).set(mfu)

    # Gradient norm (must be called BEFORE optimizer step in real code)
    gnorm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    gradient_norm.labels(job_id=job_id).set(gnorm.item())

    return loss.item(), mfu

# Start metrics server on port 8001 (Prometheus scrapes this)
start_http_server(8001)
```

17.4.2 PyTorch Profiler — Operator-Level Decomposition

```python
```

```python
import torch.profiler

def profile_training_step(model, batch, optimizer, output_dir="./profiles"):
    """
    Trigger only when MFU drops unexpectedly — too expensive for always-on use.
    Use when nsys shows a specific Python-level operation dominates.
    """
    with torch.profiler.profile(
        activities=[
            torch.profiler.ProfilerActivity.CPU,
            torch.profiler.ProfilerActivity.CUDA,
        ],
        schedule=torch.profiler.schedule(
            wait=1,    # Skip 1 warmup step
            warmup=1,  # 1 warmup step
            active=3,  # Profile 3 active steps
            repeat=1,
        ),
        on_trace_ready=torch.profiler.tensorboard_trace_handler(output_dir),
        record_shapes=True,
        profile_memory=True,
        with_stack=True,
    ) as prof:
        for step in range(5):
            loss = model(**batch).loss
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
            prof.step()

    # Print top-10 operators by CUDA time:
    print(prof.key_averages().table(
        sort_by="cuda_time_total", row_limit=10
    ))

# WHEN TO TRIGGER torch.profiler:
# Condition: training_mfu drops > 10% from rolling baseline for > 5 consecutive step
# Action: capture 5-step profile → upload to shared artifact storage → page engineer
```

17.4.3 System Metrics — Prometheus Configuration

```yaml
```

```yaml
# prometheus.yml — Complete AI cluster scrape configuration

global:
  scrape_interval:     15s      # Collect every 15 seconds
  evaluation_interval: 15s      # Evaluate alert rules every 15s
  external_labels:
    cluster: 'h100-prod-cluster'
    region: 'us-east-1'

scrape_configs:
  # GPU silicon metrics (DCGM exporter on every GPU node)
  - job_name: 'dcgm'
    static_configs:
      - targets: ['gpu-node-001:9400', 'gpu-node-002:9400']
    # Or use service discovery for dynamic clusters:
    # kubernetes_sd_configs: [{role: pod}]
    # relabel_configs: keep only pods with app=dcgm-exporter

  # OS metrics (node_exporter: CPU, RAM, disk, NIC)
  - job_name: 'node'
    static_configs:
      - targets: ['gpu-node-001:9100', 'gpu-node-002:9100']

  # Training framework metrics (custom Python Prometheus client, port 8001)
  - job_name: 'training'
    static_configs:
      - targets: ['gpu-node-001:8001']
    metrics_path: '/metrics'

  # vLLM inference metrics
  - job_name: 'vllm'
    static_configs:
      - targets: ['inference-server-001:8000']
    metrics_path: '/metrics'

  # SLURM scheduler metrics (slurm_exporter)
  - job_name: 'slurm'
    static_configs:
      - targets: ['slurm-head:9341']

  # InfiniBand NIC health (infiniband_exporter)
  - job_name: 'infiniband'
    static_configs:
```

```yaml
      - targets: ['gpu-node-001:9315']

rule_files:
  - "gpu_alerts.yml"
  - "inference_alerts.yml"
  - "training_alerts.yml"

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']
```

17.5 Layer 5 — Inference Service Metrics

17.5.1 vLLM Prometheus Metrics Reference

```
VLLM PROMETHEUS METRICS — COMPLETE REFERENCE:

  Metric                                    Type       Alert Threshold
  ──────────────────────────────────────────────────────────────────────────
  vllm:num_requests_running                 Gauge      > 256 (KV pressure)
  vllm:num_requests_waiting                 Gauge      > 20 (queue growing)
  vllm:gpu_cache_usage_perc                Gauge      > 0.82 (admission risk)
  vllm:num_preemptions_total               Counter    rate > 0 (KV evictions)
  vllm:time_to_first_token_seconds         Histogram  P99 > SLA target
  vllm:time_per_output_token_seconds       Histogram  P99 > SLA target
  vllm:e2e_request_latency_seconds         Histogram  P99 > SLA target
  vllm:request_prompt_tokens               Histogram  Track distribution
  vllm:request_generation_tokens           Histogram  Track distribution
  vllm:spec_decode_draft_acceptance_rate   Gauge      < 0.60 (disable spec)
  ──────────────────────────────────────────────────────────────────────────


KEY PROMQL QUERIES FOR INFERENCE DASHBOARDS:

  # TTFT P99 (5-minute window)
  histogram_quantile(0.99,
    rate(vllm:time_to_first_token_seconds_bucket[5m])
  )

  # TPOT P50 and P99
  histogram_quantile(0.50, rate(vllm:time_per_output_token_seconds_bucket[5m]))
```

```
  histogram_quantile(0.99, rate(vllm:time_per_output_token_seconds_bucket[5m]))

  # Decode throughput (tokens per second, global)
  sum(rate(vllm:request_generation_tokens_sum[1m]))

  # KV cache pressure (admission control trigger)
  vllm:gpu_cache_usage_perc > 0.82

  # Request queue depth alert
  vllm:num_requests_waiting > 20

  # GPU utilization for serving nodes (DCGM correlated)
  avg by (Hostname) (DCGM_FI_DEV_GPU_UTIL{job="dcgm"})
    # Should be 65-80% for healthy interactive serving
    # > 85% sustained → queue building, SLA at risk
```

17.5.2 Distributed Tracing for Multi-Service Inference

```
DISTRIBUTED TRACING — THE MISSING LAYER:

  Why Prometheus alone is insufficient for inference SLA diagnosis:

  TTFT = t_load_balancer + t_tokenizer + t_queue + t_prefill + t_first_decode

  Each component is a separate service. Prometheus shows per-service metrics.
  But it cannot show: "Why did THIS specific request take 8 seconds?"

  Distributed tracing (OpenTelemetry + Jaeger/Zipkin) creates a SPAN for each
  service boundary in the request critical path:

  [LB: 2ms] → [Auth: 12ms] → [Queue wait: 1,800ms] → [Prefill: 320ms] → ...
                                     ^
                                     This is the problem — visible in trace, not
Prometheus

  OPENTELEMETRY INTEGRATION (vLLM / FastAPI):
    from opentelemetry import trace
    from opentelemetry.exporter.jaeger.thrift import JaegerExporter
    from opentelemetry.sdk.trace import TracerProvider
    from opentelemetry.sdk.trace.export import BatchSpanProcessor

    tracer_provider = TracerProvider()
    jaeger_exporter = JaegerExporter(
```

```
        agent_host_name="jaeger-agent",
        agent_port=6831,
    )
    tracer_provider.add_span_processor(BatchSpanProcessor(jaeger_exporter))
    trace.set_tracer_provider(tracer_provider)
    tracer = trace.get_tracer("llm-serving")

    @app.post("/v1/completions")
    async def generate(request: CompletionRequest):
        with tracer.start_as_current_span("request_total") as span:
            span.set_attribute("prompt_tokens", len(request.prompt.split()))
            with tracer.start_as_current_span("queue_wait"):
                batch_slot = await scheduler.acquire_slot()
            with tracer.start_as_current_span("prefill"):
                kv_cache = await engine.prefill(request)
            with tracer.start_as_current_span("decode"):
                return await engine.decode(kv_cache, request)

  WHEN TO DEPLOY DISTRIBUTED TRACING:
    Any inference system with > 3 microservices (tokenizer, scheduler, model,
cache)
    When TTFT P99 >> TTFT P50 and the cause is unclear
    Before going to production (add it from day one — retrofitting is hard)
```

17.6 Layer 6 — Prometheus, Grafana, and Fleet Alerting [r5]

17.6.1 Alert Rules

```yaml
```

```yaml
# gpu_alerts.yml — Production GPU cluster alert rules

groups:
  - name: gpu_hardware_health
    rules:
      # CRITICAL: Uncorrectable ECC — GPU must be drained immediately
      - alert: GPUUncorrectableECC
        expr: DCGM_FI_DEV_ECC_DBE_VOL_TOTAL > 0
        for: 0m                          # Page immediately — no grace period
        labels:
          severity: critical
          team: infrastructure
        annotations:
          summary: "Uncorrectable ECC on {{ $labels.Hostname }} GPU {{ $labels.gpu }}
          action: "Drain node immediately. File GPU replacement ticket."

      # WARNING: Correctable ECC rate — schedule replacement
      - alert: GPUCorrectableECCRising
        expr: rate(DCGM_FI_DEV_ECC_SBE_VOL_TOTAL[1h]) > 5
        for: 10m
        labels:
          severity: warning
          team: infrastructure
        annotations:
          summary: "High ECC correctable rate: {{ $value | humanize }}/hr"
          action: "Schedule GPU replacement within 48h."

      # CRITICAL: Thermal throttle engaged
      - alert: GPUThermalThrottle
        expr: DCGM_FI_DEV_THERMAL_VIOLATION > 0
        for: 1m
        labels:
          severity: critical
          team: infrastructure
        annotations:
          summary: "GPU thermal throttle on {{ $labels.Hostname }}"
          action: "Check cooling system. Reduce power cap if needed."

      # WARNING: Temperature approaching throttle threshold
      - alert: GPUTemperatureHigh
        expr: DCGM_FI_DEV_GPU_TEMP > 80
        for: 5m
        labels:
```

```yaml
        severity: warning
      annotations:
        summary: "GPU temp {{ $value }}°C on {{ $labels.Hostname }}"

    # WARNING: SM clock below boost baseline (throttle indicator)
    - alert: GPUClockThrottle
      expr: DCGM_FI_DEV_SM_CLOCK < 1700
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "GPU clock degraded: {{ $value }} MHz (expected 1785-1980)"

  - name: training_efficiency
    rules:
    # WARNING: MFU dropped significantly
    - alert: TrainingMFUDrop
      expr: |
        training_mfu < (
          avg_over_time(training_mfu[30m] offset 30m) * 0.9
        )
      for: 10m
      labels:
        severity: warning
        team: ml_ops
      annotations:
        summary: "Training MFU dropped >10% from 30-min baseline"
        action: "Check NCCL overlap, data loading, and GPU health."

    # WARNING: Gradient norm spike
    - alert: GradientNormSpike
      expr: training_gradient_norm > 10
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: "Gradient norm spike: {{ $value }} (expected <2)"

    # CRITICAL: NaN gradient norm
    - alert: GradientNormNaN
      expr: training_gradient_norm == NaN or training_gradient_norm == Inf
      for: 0m
      labels:
        severity: critical
```

```yaml
        annotations:
          summary: "NaN/Inf gradient norm detected — training diverged"
          action: "Suspend training. Check for corrupted checkpoints or data."

  - name: inference_sla
    rules:
      # WARNING: TTFT P99 approaching SLA
      - alert: InferenceTTFTP99High
        expr: |
          histogram_quantile(0.99,
            rate(vllm:time_to_first_token_seconds_bucket[5m])
          ) > 0.6
        for: 3m
        labels:
          severity: warning
        annotations:
          summary: "TTFT P99 = {{ $value | humanize }}s"

      # CRITICAL: KV cache admission risk
      - alert: KVCacheHighPressure
        expr: vllm:gpu_cache_usage_perc > 0.82
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "KV cache at {{ $value | humanizePercentage }}"
          action: "Enable admission control at 82% threshold."

      # WARNING: KV cache evictions occurring
      - alert: KVCacheEvictions
        expr: rate(vllm:num_preemptions_total[5m]) > 0
        for: 1m
        labels:
          severity: warning
        annotations:
          summary: "KV evictions occurring: {{ $value | humanize }}/sec"
```

17.6.2 Grafana Dashboard Design [r5]

```
ESSENTIAL GRAFANA DASHBOARDS FOR AI CLUSTER:

DASHBOARD 1 — FLEET HEALTH HEATMAP (most used):
  Panel: GPU Utilization Heatmap (hostname × time, color = SM util %)
```

```
  PromQL: DCGM_FI_DEV_GPU_UTIL
  Color: green (>70%) → yellow (40-70%) → red (<40%)
  Purpose: Instantly see idle GPUs, cold GPUs, straggler nodes

  Panel: ECC Error Rate Timeline (per node)
  PromQL: rate(DCGM_FI_DEV_ECC_SBE_VOL_TOTAL[1h])

  Panel: Temperature Map (per GPU, all nodes)
  PromQL: max by (Hostname) (DCGM_FI_DEV_GPU_TEMP)
  Threshold annotations: 78°C warning, 83°C critical

DASHBOARD 2 — INFERENCE SLA:
  Panel: TTFT Percentiles (P50, P95, P99) — time series
  PromQL: histogram_quantile(0.{50,95,99},
rate(vllm:time_to_first_token_seconds_bucket[5m]))

  Panel: KV Cache Utilization — single stat gauge + trend
  PromQL: avg(vllm:gpu_cache_usage_perc)
  Thresholds: 0.80 = yellow, 0.88 = red (cascade risk zone)

  Panel: Queue Depth — bar chart per second
  PromQL: vllm:num_requests_waiting

  Panel: Throughput (tokens/sec) — time series
  PromQL: sum(rate(vllm:request_generation_tokens_sum[1m]))

DASHBOARD 3 — TRAINING EFFICIENCY:
  Panel: MFU per Job — time series (one line per active job)
  PromQL: training_mfu

  Panel: Step Time Distribution — P50/P99 percentile bands
  PromQL: histogram_quantile(0.{50,99},
rate(training_step_time_seconds_bucket[5m]))

  Panel: AllReduce Fraction — bar chart
  PromQL: avg(rate(training_allreduce_time_seconds_sum[5m])) /
avg(rate(training_step_time_seconds_sum[5m]))
  Threshold: > 0.20 (20% of step time) = problem

  Panel: Gradient Norm Trend — time series with anomaly bands
  PromQL: training_gradient_norm

RECORDING RULES (pre-compute expensive queries for dashboard performance):
  # Pre-compute every 60s to make dashboards instant:
```

```
  record: fleet:gpu_util:avg
  expr:   avg(DCGM_FI_DEV_GPU_UTIL)

  record: fleet:tensor_active:avg
  expr:   avg(DCGM_FI_PROF_PIPE_TENSOR_ACTIVE)

  record: serving:ttft_p99:5m
  expr:   histogram_quantile(0.99,
rate(vllm:time_to_first_token_seconds_bucket[5m]))

  record: fleet:gpu_above_82c:count
  expr:   count(DCGM_FI_DEV_GPU_TEMP > 82)
```

17.7 Straggler Detection and Anomaly Patterns [r5]

17.7.1 The Three Straggler Signals

```
STRAGGLER DETECTION VIA PROMETHEUS — THREE SIGNALS:

SIGNAL 1 — COMPUTE STRAGGLER (utilization outlier):
  A single GPU running below its peers when all should be compute-saturated.

  Detection query:
    DCGM_FI_DEV_GPU_UTIL{Hostname="$node"}
    < (avg by(Hostname)(DCGM_FI_DEV_GPU_UTIL) - 20)

  Alert rule:
    expr: |
      (DCGM_FI_DEV_GPU_UTIL) <
      (avg by(Hostname)(DCGM_FI_DEV_GPU_UTIL) - 20)
    for: 5m
    # If GPU-0 is at 45% while GPU-1 through GPU-7 are at 85%:
    # GPU-0 is straggling — holding back the AllReduce ring for all 8 GPUs.

  Root causes: hardware degradation, driver fault, memory allocation stall

SIGNAL 2 — THERMAL STRAGGLER (clock throttle):
  SM clock below boost baseline = thermal throttle = 10% slower computations
  → Adds 10% to every AllReduce barrier for the entire 8-GPU ring

  Detection:
```

```
    DCGM_FI_DEV_SM_CLOCK < 1700 AND DCGM_FI_DEV_GPU_TEMP > 78

  Cross-correlation (confirm thermal cause):
    DCGM_FI_DEV_THERMAL_VIOLATION > 0 AND
    DCGM_FI_DEV_SM_CLOCK < expected_boost AND
    DCGM_FI_DEV_GPU_TEMP > 78

  Root causes: blocked airflow, DLC pump failure, coolant temperature rise

SIGNAL 3 — NETWORK STRAGGLER (IB link degradation):
  A node with a degraded IB link contributes its AllReduce packets late.
  All other nodes wait at the ring barrier.

  Detection:
    rate(infiniband_port_receive_errors_total{port="$port"}[5m]) > 0
    OR
    infiniband_port_link_speed_bytes{port="$port"} < 47000000000 # Below 376 Gbps

  Cross-reference with step-time variance:
    P99 step time > 1.15× P50 step time sustained → straggler present
    Correlate with IB error counters → identify which node

INVESTIGATION WORKFLOW:
  1. Grafana: GPU utilization heatmap → spot outlier GPU
  2. DCGM: check SM clock, thermal violation, ECC rate on outlier GPU
  3. If thermal: check rack cooling, adjacent temperatures, DLC flow
  4. If hardware (ECC + low perf): drain node, schedule GPU replacement
  5. If IB: check switch port error counters (ibstatus), NIC health (ibdiag)
  6. If software: check for PS/kernel preemptions, memory pressure, OOM signals
```

17.7.2 KV Cache Cascade — The Inference Cliff

```
KV CACHE CASCADE FAILURE — THE 90% CLIFF:

  Normal operation (KV util < 80%):
    New request arrives → allocate N blocks → serve normally
    Old request completes → free N blocks → steady state

  At 90% KV utilization (the cliff):
    New request arrives → needs N blocks
    Only N-1 blocks free → must evict an existing sequence
    Eviction of sequence = free its blocks = but that sequence is now
    invalid and must be recomputed from scratch when it comes back
```

```
    EXPECTED CASCADE LENGTH (queuing theory):
      If each allocation triggers on average 0.9 additional evictions:
      E[cascade] = 1 / (1 - 0.9) = 10 evictions per new allocation
      Each eviction adds ~12ms (PCIe DMA for 600MB sequence)
      10 cascades × 12ms = 120ms added to TTFT for new request
      This compounds across all requests in the batch.

    P99 TTFT can spike 10-50× in seconds above the 90% KV threshold.

  ADMISSION CONTROL RULE:
    Alert at 82% KV utilization → page on-call
    Admission control threshold at 82% → return HTTP 429
    Never operate above 85% sustained.

    PromQL alert (fires BEFORE cascade begins):
      vllm:gpu_cache_usage_perc > 0.82

  THE KV MONITORING CRITICAL INSIGHT:
    KV util > 88% alert threshold is TOO LATE.
    By the time the 88% alert fires, cascading evictions have already started.
    The 82% alert fires with 6-8 percentage points of headroom — time to act.
    This headroom is consumed in < 60 seconds during traffic bursts.
    Set the alert at 82%, not 90%.
```

17.8 Layer 7 — Performance Regression CI Pipeline

17.8.1 Regression Detection Methodology

```
STATISTICAL BASELINES, NOT FIXED THRESHOLDS:

  Problem with fixed thresholds:
    Alert if MFU < 45% misses a 10% regression from a 40% baseline.
    Alert if MFU < 45% generates false alerts for a new lower-MFU workload.

  CORRECT APPROACH: Rolling mean and standard deviation per workload type.
    baseline_mean = rolling_7day_avg(training_mfu, workload_id)
    baseline_std  = rolling_7day_std(training_mfu, workload_id)

    Alert when z-score < -3:
      z = (current_mfu - baseline_mean) / baseline_std < -3.0
```

```
    This approach:
      - Robust to workload changes (baseline updates automatically)
      - Sensitive to genuine regressions (3σ threshold = 0.3% false-positive rate)
      - Per-workload granularity (different baselines for different models/batch
sizes)

REGRESSION CI PIPELINE — THREE TIERS:

  TIER 1: Every commit to main/release branch (< 5 minutes):
    GEMM TFLOPS benchmark (3 shapes)        Expected: < 3% deviation
    HBM bandwidth test (1 GB transfer)      Expected: > 85% of baseline
    FlashAttention forward (batch=8, S=2K)  Expected: < 3% latency deviation
    Kernel launch overhead                  Expected: < 5 μs per launch

  TIER 2: Driver / framework version bump (< 20 minutes):
    All Tier 1 tests
    LLM prefill/decode sweep (S=128 to 4K)  Expected: < 3% TTFT P95 regression
    Single-GPU training step (100 steps)    Expected: < 3% step time regression
    MFU validation (compare to 7-day rolling baseline, alert at > 3σ)

  TIER 3: Nightly release gate (1-2 hours):
    All Tier 1 + Tier 2 tests
    Multi-GPU AllReduce (8-GPU, 256MB-4GB)  Expected: < 5% busbw regression
    End-to-end training MFU (100 steps)     Expected: > 55% MFU, < 3% regression
    Inference throughput-latency curve      Expected: < 5% P99 TTFT regression

AUTO-GENERATE NSIGHT TRACE ON REGRESSION:
  When any P0/P1 regression detected:
    1. Extract regressing kernel name from benchmark report
    2. Auto-run: ncu --set full --kernel-name <kernel> python benchmark.py
    3. Upload .ncu-rep artifact + regression diff to incident ticket
    4. Post PR comment with: % regression, metric name, ncu artifact link
    5. BLOCK merge for P0/P1; WARN (no block) for P2
```

17.8.2 Alert Priority and Response Matrix

```
ALERT PRIORITY MATRIX:

  Priority  Metric                    Threshold      Action
  ──────────────────────────────────────────────────────────────────────────
  P0        Training MFU              > 5% drop       Block merge; auto-page
oncall
```

```
  P0        GEMM TFLOPS               > 5% drop       Block merge; auto-page
oncall
  P0        HBM bandwidth             > 5% drop       Block merge; hardware check
  P0        GPU uncorrectable ECC     ANY              Drain node; hardware
replacement
  P0        Training gradient = NaN   ANY              Suspend job; checkpoint
rollback
  P1        TTFT P99                  > 5% increase   Block merge; notify author
  P1        TPOT P99                  > 5% increase   Block merge; notify author
  P1        KV cache > 82%            Sustained 2min  Enable admission control;
page
  P1        SM clock throttle         < 1700 MHz       Investigate cooling
  P1        GPU temperature           > 82°C           Check cooling; reduce power
cap
  P2        AllReduce bandwidth       > 5% drop        Warning in PR; investigate
  P2        Scaling efficiency        > 3% drop        Warning in PR
  P2        ECC correctable rate      > 5/hour         Schedule replacement 48h
  ──────────────────────────────────────────────────────────────────────────────
```

17.9 AMD Observability Stack

```
AMD TOOL MAPPING (MI300X / CDNA architecture):

  Layer     NVIDIA                        AMD
  ──────────────────────────────────────────────────────────────────────────

  Silicon   DCGM + dcgm-exporter          AMDSMI + amd-smi-exporter
            (DCGM_FI_DEV_* fields)        (amd_smi_gpu_utilization, etc.)

  Kernel    Nsight Compute                Omniperf / ROCprofiler
  (single)  (ncu --kernel-name)           (omniperf profile --name workload)

  Timeline  Nsight Systems                ROCprofiler-systems (rocprof-sys)
            (nsys profile)                (rocprofiler-sdk-tool)

  Framework PyTorch Profiler              PyTorch Profiler (same — cross-platform)

  System    Prometheus + node_exporter    Prometheus + node_exporter (same)

  Fleet     Grafana + PagerDuty           Grafana + PagerDuty (same)
  ──────────────────────────────────────────────────────────────────────────
```

```
AMD-SPECIFIC COMMANDS:
  # MI300X GPU health check:
  amd-smi monitor --watch 1 --watch-time 30    # 30-second health monitor
  amd-smi metric --gpu all --mem               # Memory utilization

  # ROCprofiler system trace (equivalent to nsys):
  rocprofiler-sdk-tool profile --hip-trace -- python train.py

  # Omniperf roofline (equivalent to ncu Roofline):
  omniperf profile --name workload_name -- python train.py
  omniperf analyze --path workload_name/ --roof-only

  # AMD SMI Prometheus exporter (deploy as DaemonSet, similar to dcgm-exporter):
  # Port: 2021 by default
  # Key metrics: amd_smi_gpu_utilization, amd_smi_memory_temperature,
  #              amd_smi_pcie_bandwidth, amd_smi_power_usage

NOTE: Framework metrics (MFU logging, step time, gradient norm) and fleet-level
tools (Prometheus, Grafana, alerting) are identical between NVIDIA and AMD
clusters.
The silicon + kernel profiling layers are where the stacks diverge.
```

17.10 Key Takeaways and Review Questions

```
OBSERVABILITY KEY PRINCIPLES (r3 integration):

  1. DCGM + PROMETHEUS + GRAFANA IS THE BASELINE — DEPLOY BEFORE YOU NEED IT:
     Most organizations deploy monitoring after problems occur.
     The correct approach: monitoring is day-zero infrastructure.
     A GPU that starts throttling on day 1 without monitoring wastes every
     subsequent day of training. Deploy DCGM exporter before submitting job 1.

  2. CORRECTABLE ECC IS A LEADING INDICATOR, NOT A HARMLESS EVENT:
     GPU with 5 SBE/hour: ~1% chance of uncorrectable error within 24h.
     Pre-emptive replacement: costs 1 maintenance window.
     Post-failure replacement: costs the checkpoint-to-failure GPU-hours.
     Alert on correctable ECC trends. Page immediately on any uncorrectable.

  3. STATISTICAL BASELINES, NOT FIXED THRESHOLDS, FOR REGRESSION DETECTION:
     Fixed "alert if MFU < 45%": misses 10% regression from 40% baseline.
     Rolling 7-day mean ± 3σ: catches genuine regressions, survives workload
```

```
changes.
     This approach is robust to new models and sensitive to driver regressions.

  4. CI REGRESSION PIPELINE IS THE INFRASTRUCTURE THAT PROTECTS YOUR
OPTIMIZATIONS:
     A driver update can silently regress MFU by 8-15%.
     Without CI: discovered weeks later after millions of wasted GPU-hours.
     With CI: caught in 20-minute Tier 2 benchmark, blocked before fleet
deployment.
     The CI pipeline costs ~$2K/month to run; saves $50-200K per caught
regression.

  5. NSIGHT SYSTEMS SCOPES THE PROBLEM; NSIGHT COMPUTE DIAGNOSES IT:
     Running ncu on a full training step = 100× slowdown, 10+ GB of data.
     Always nsys first (5 minutes, 2-3× overhead) → identify bottleneck kernel.
     Then ncu --kernel-name (20 minutes, targeted) → diagnose why.
     Two-stage protocol: reduces investigation time from days to hours.

  6. DISTRIBUTED TRACING IS THE MISSING LAYER FOR INFERENCE:
     DCGM tells you a GPU is slow.
     Prometheus tells you TTFT P99 is high.
     Distributed tracing (OpenTelemetry + Jaeger) tells you WHICH component
     in the 5-service inference pipeline caused the 8-second TTFT.
     Deploy from day one. Retrofitting distributed tracing is painful.

TOOL QUICK REFERENCE:
  "Where is time going?"         nsys profile --trace=cuda,nvtx,nccl
  "Why is THIS kernel slow?"     ncu --set full --kernel-name X
  "Is GPU healthy (fleet)?"      dcgmi dmon -e 203,155,100,1001,1002
  "Is AllReduce overlapped?"     nsys with NCCL trace (look at NCCL stream
position)
  "Is IB healthy?"               ibstatus all nodes; infiniband_exporter metrics
  "Is training MFU OK?"          training_mfu Prometheus gauge + WandB/MLflow
  "Is inference SLA met?"        histogram_quantile(0.99,
vllm:time_to_first_token)
  "Which inference request slow?" OpenTelemetry trace in Jaeger
```

Review Questions

Difficulty: Intermediate

Question 1 (Observability Gap Analysis): A 2,048-GPU training run has been running for 12 days at 48% MFU. The deployed observability: DCGM per-GPU metrics to Prometheus, loss/LR curves to W&B, and SLURM job accounting. On day 13, the training lead reports training "seems

slower." (a) List three specific metrics NOT collected that would immediately diagnose the problem. (b) Without new tooling, what is the fastest way to diagnose the slowdown using existing tools? (c) Design the minimal additional telemetry that would have surfaced this problem automatically within 10 minutes of onset.

Expected answer: (a) Three missing metrics: (1) `training_step_time_seconds` per step (wall clock per training step — not available from DCGM or SLURM accounting, requires custom Prometheus gauge in the training loop). Without this, "seems slower" cannot be quantified — is it 5% or 30%? (2) `training_allreduce_time_seconds` (time spent in gradient AllReduce per step — isolates whether communication is the bottleneck). (3) Per-step DataLoader wait time (InstrumentedDataLoader avg_wait_ms from Chapter 15). These three together triangulate: compute, communication, or data loading. (b) Fastest diagnosis with existing tools: `dcgmi dmon -e 203,1003,1009 -d 1` on a few nodes simultaneously. DCGM field 203 (SM util) — if SM util is high: compute or communication bottleneck. Field 1009 (NVLink TX bytes) — if NVLink bandwidth has dropped: TP AllReduce degraded. Cross-reference with DCGM_FI_DEV_SM_CLOCK (field 150) — if clock is below boost: thermal throttle. Check SLURM squeue for job step timings — if step time distribution has widened (P99 >> P50): straggler node. Also: `NCCL_DEBUG=INFO` on one rank for 10 minutes to see AllReduce timing. (c) Minimal additional telemetry for 10-minute auto-detection: add one custom Prometheus gauge in the training loop (10 lines of Python): `training_step_time_seconds.observe(step_duration)` and `training_mfu.set(computed_mfu)`. Alert rule: `training_mfu < (avg_over_time(training_mfu[30m] offset 30m) * 0.90) for 10m` → this fires within 10 minutes of any 10%+ MFU regression and points directly to the affected job. Total additional engineering time: ~2 hours for instrumentation + 30 minutes for alert rule.

Difficulty: Principal Level

Question 2 (Inference SLA Regression Root Cause): Your LLaMA-3 70B inference cluster shows: TTFT P50 = 410ms (normal), TTFT P99 = 8,200ms (severely elevated; SLA is < 800ms). TPOT P50 and P99 are both normal. GPU utilization (DCGM_FI_DEV_GPU_UTIL) is 94% sustained. Using the observability stack, diagnose the root cause and prescribe the fix.

Expected answer: TTFT P99 >> TTFT P50 while TPOT is normal means: DECODE is working fine (TPOT is the decode metric) but FIRST TOKEN delivery is suffering for the 1% of requests. The P99/P50 ratio of 20× is extreme — this is not gradual degradation, it is a cliff effect. Combined with 94% GPU utilization: the GPU is busy, not idle. This is NOT a compute bottleneck. This is a queueing problem. Diagnosis: vLLM metric `vllm:gpu_cache_usage_perc` > 0.88 (the cascade eviction zone). At 94% GPU utilization and high TTFT P99, the most likely cause is KV cache pressure causing cascading evictions. Each eviction of a long-context request frees blocks that are immediately claimed by new requests → cascade → P99 TTFT spikes 20×. Confirming evidence: check `vllm:num_preemptions_total` rate > 0. Also check

`vllm:num_requests_waiting` — if queue > 20 requests: admission control threshold was already breached. Root cause: KV admission control threshold not set (or set too high at 90% instead of 82%). Fix: (1) Immediate: enable admission control: `--kv-cache-threshold 0.82` or custom middleware returning HTTP 429 at 82% KV utilization. (2) Medium-term: enable FP8 KV cache (halves KV memory → doubles available sequences before hitting 82%). (3) If FP8 KV doesn't resolve: add more serving replicas.

Difficulty: Principal Level

Question 3 (Dashboard Design for Production Fleet): Design the minimum viable Grafana dashboard suite for a production cluster running: 4,096 GPU training jobs, 200 inference serving instances. Specify exactly: panels, PromQL queries, and alert thresholds for each.

Expected answer: Three-dashboard minimum:

Dashboard 1 — Fleet Hardware Health: Panel 1: GPU Utilization Heatmap. PromQL: `DCGM_FI_DEV_GPU_UTIL`. Heatmap: x=time, y=hostname, color=util%. Thresholds: green (>70%), yellow (40-70%), red (<40% during active job). Purpose: instantly spot idle GPUs, thermal events, straggler nodes. Panel 2: ECC Error Rate. PromQL: `rate(DCGM_FI_DEV_ECC_SBE_VOL_TOTAL[1h])`. Time series. Alert: > 5/hr = warning; any DBE = critical. Panel 3: Temperature Map. PromQL: `max by(Hostname)(DCGM_FI_DEV_GPU_TEMP)`. Color threshold: 78°C=yellow, 82°C=red. Panel 4: Drained Node Count. PromQL: `slurm_nodes_states{state="drained"}`. Single stat. Alert at > 5% of cluster.

Dashboard 2 — Training Efficiency: Panel 1: MFU per Job. PromQL: `training_mfu`. Multi-line time series. Alert: any job dropping > 10% from 30-min baseline. Panel 2: Step Time P50/P99. PromQL: `histogram_quantile(0.{50,99}, rate(training_step_time_seconds_bucket[5m]))`. Band chart. Alert: P99 > 1.5× P50 sustained 10min. Panel 3: AllReduce Fraction. PromQL: `avg(rate(training_allreduce_time_seconds_sum[5m])) / avg(rate(training_step_time_seconds_sum[5m]))`. Alert: > 0.20 (20% of step time).

Dashboard 3 — Inference SLA: Panel 1: TTFT P50/P99. PromQL: `histogram_quantile(0.{50,99}, rate(vllm:time_to_first_token_seconds_bucket[5m]))`. Alert: P99 > SLA target × 1.5. Panel 2: KV Cache Pressure. PromQL: `avg(vllm:gpu_cache_usage_perc)`. Gauge + trend. Threshold: 0.82 red (admission control trigger). Panel 3: Throughput. PromQL: `sum(rate(vllm:request_generation_tokens_sum[1m]))`. Target line: expected tokens/sec for fleet size. Panel 4: Queue Depth. PromQL: `sum(vllm:num_requests_waiting)`. Alert: > 20.

Difficulty: Principal Level

Question 4 (Silent Data Corruption Detection): You are designing the SDC (Silent Data Corruption) detection layer for a 16,384-GPU training cluster. SDC events occur at roughly 1 per 10,000 GPU-months. (a) Calculate the expected SDC frequency for this cluster during a 90-day run. (b) Describe the four-layer detection hierarchy with the specific metric at each layer, its detection latency, and false-positive rate. (c) An SDC is suspected at step 48,200 of a 100,000-step run. Last clean checkpoint is step 47,500. How do you validate the checkpoint at step 47,500?

Expected answer: (a) Expected SDCs: 16,384 GPUs × (90/30) months × (1/10,000) = 16,384 × 3 / 10,000 = 4.92 expected SDC events over the 90-day run. Approximately 5 SDC events are expected — not negligible. Even at 1 event, a corrupted weight tensor propagates silently through the model, degrading accuracy without any visible crash or error. (b) Four-layer detection hierarchy: Layer 1 (Hardware): DCGM ECC correctable errors (DCGM_FI_DEV_ECC_SBE_VOL_TOTAL). Latency: immediate (1-second sampling). False positives: low (ECC hardware detects accurately). Limitation: ECC corrects most bit flips — SDC means it was NOT caught by ECC. Layer 2 (Gradient): gradient norm monitoring. SDC in weights → abnormal gradient magnitudes → `training_gradient_norm` spike. Latency: next backward pass (seconds to minutes). False positives: moderate (legitimate spikes from bad data batches). Layer 3 (Loss): training loss curve anomaly. SDC that doesn't cause gradient spike may still cause loss plateau or divergence. Latency: 100-1000 steps (minutes to hours). False positives: low (clear loss divergence pattern). Layer 4 (Validation): periodic validation set evaluation. Run eval every N steps; compare to previous eval checkpoint. Latency: hours to days. False positives: very low. (c) Checkpoint validation at step 47,500: (1) Run the validation benchmark on the step 47,500 checkpoint. Compare validation loss to the logged value at step 47,500 from W&B/MLflow. If they match within 0.5%: checkpoint is clean. (2) Check the gradient norm logged at steps 47,500 to 48,200. If any spike was visible in the gradient norm time series during this window: the corruption may have started before 47,500. In that case: roll back to 47,000 or earlier and repeat validation. (3) Optionally: run the step 47,500 → 47,510 forward pass on a clean node with the same data batch (deterministic with fixed seed) and compare activations at a checkpoint layer to what was logged. Matching activations confirm the weights at 47,500 are clean. (4) If validation confirms step 47,500 is clean: resume from there, losing only 700 steps (~0.7% of the 100K-step run).

Further Reading

1. *Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems* — Gunawi et al. (FAST 2018). `usenix.org/conference/fast18/presentation/gunawi` The empirical basis for ECC-trend alerting and proactive node replacement.
2. *NVIDIA DCGM User Guide* — NVIDIA (living document). `docs.nvidia.com/datacenter/dcgm/latest/user-guide/` Chapters 4 (Health and

Diagnostics) and 7 (Profiling Metrics) are the primary references for every field ID used in production monitoring.

3. *Understanding and Improving Failure Tolerance in Production AI Training Clusters* — Microsoft Research (2023). `arxiv.org/abs/2311.00049` Taxonomy of failure modes, detection latency distributions, and recovery time breakdown from thousands of production AI training jobs.
4. *Prometheus Documentation — Alerting and Recording Rules*. `prometheus.io/docs/prometheus/latest/configuration/alerting_rules/` The definitive reference for PromQL-based alerting syntax.
5. *NVIDIA Nsight Systems User Guide* — NVIDIA (2024). `docs.nvidia.com/nsight-systems/` CUDA trace, NCCL trace, and the GPU timeline sections.
6. *OpenTelemetry Python Documentation*. `opentelemetry-python.readthedocs.io` The distributed tracing integration guide for FastAPI and async Python services.

— End of Chapter 17 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved